Database Design

COMS3011A Lab 1 — Todo Application (SQLite)

Overview

The schema has two tables: topics and tasks, related one-to-many. Every task belongs to exactly one topic; a topic can have many tasks. Status and the overdue indicator are handled without extra tables — status is a fixed enum enforced by a CHECK constraint, and overdue is never stored, only computed at read time.

Entity-relationship diagram

```
topics (1) ——— categorises ———▶ (∞) tasks
```

Each task references exactly one topic via the topic_id foreign key. Deleting a topic with existing tasks is not supported by this schema (no ON DELETE clause is set, so SQLite's default RESTRICT behaviour applies) — the application should prevent removing a topic that is still in use.

Table: topics

Stores the fixed set of topics a task can belong to. Normalised into its own table (rather than a free-text column on tasks) so the value is consistent everywhere it's used and so sorting/grouping by topic is a simple join, not a distinct-string scan.

Column	Type	Constraints	Purpose
id	INTEGER	PK, AUTOINCREMENT	Surrogate key referenced by tasks.topic_id.
name	TEXT	NOT NULL, UNIQUE	The topic label shown in the UI; uniqueness prevents duplicate topics like "Work" and "work".

Table: tasks

Stores every task the user has created, including archived ones.

Column	Type	Constraints	Purpose
id	INTEGER	PK, AUTOINCREMENT	Surrogate key for the task.
title	TEXT	NOT NULL	Required task field.
description	TEXT	nullable	Optional task field.
due_date	TEXT	NOT NULL	ISO-8601 date (e.g. 2026-08-04). Stored as text since SQLite has no native date type; ISO-8601 sorts correctly as a string, which drives the “sort by due date” view.
topic_id	INTEGER	NOT NULL, FK → topics.id	Links the task to exactly one topic.
status	TEXT	NOT NULL, DEFAULT 'todo', CHECK IN ('todo','in_progress','complete')	Fixed, non-user-customisable status. A CHECK constraint enforces the three allowed values without a lookup table.
archived_at	TEXT	nullable	NULL = active/visible in the main list. A timestamp = archived, and when. Tasks are never deleted, so archiving is a flag, not a row removal.

Column	Type	Constraints	Purpose
created_at	TEXT	NOT NULL, DEFAULT now	Row creation time, for audit/ordering.
updated_at	TEXT	NOT NULL, DEFAULT now	Last-modified time; the app should update this on every edit.

Design decisions

- Archiving is a flag, not a delete or a copy. archived_at is nullable: NULL means active, a timestamp means archived. This satisfies the requirement that a task “cannot be deleted, only archived, so that it remains viewable” — the row never moves or disappears, the active list simply filters on archived_at IS NULL.
- Status is a CHECK constraint, not a lookup table. The brief fixes the three statuses (Todo, In-Progress, Complete) as non-user-customisable, so a separate statuses table would add a join for no benefit. CHECK (status IN (...)) gives the same integrity guarantee directly on the column.
- Overdue is derived, never stored. It is not a column, and it is not a fourth status value — the brief is explicit that overdue must be shown “but not as a status.” It's computed at query time and does not depend on archived state, since archived tasks leave the active list and are no longer subject to the overdue rule shown there.
- Dates are stored as ISO-8601 text. SQLite has no native date/time type; ISO-8601 strings (YYYY-MM-DD, or full timestamps for created_at/updated_at) sort and compare correctly as plain text, which is what the due-date sort and the overdue comparison rely on.
- topic_id is NOT NULL. Every task must belong to a topic — there is no implicit “uncategorised” bucket. If that's ever needed, seed a default topic row rather than allowing NULL, so sorting/grouping by topic doesn't need a null-handling special case.

Deriving “overdue” at read time

```
SELECT *,        (status != 'complete'         AND archived_at IS NULL         AND
due_date < date('now')) AS is_overdue FROM tasks;
```

A task counts as overdue only while it is active (not archived) and not already complete. Once a task is archived, it is out of the active list and the overdue flag no longer applies to it there — consistent with overdue never being a selectable status.

Full DDL

```
CREATE TABLE topics (     id   INTEGER PRIMARY KEY AUTOINCREMENT,     name TEXT NOT
NULL UNIQUE );  CREATE TABLE tasks (     id           INTEGER PRIMARY KEY
AUTOINCREMENT,     title        TEXT NOT NULL,     description  TEXT,     due_date
TEXT NOT NULL,     topic_id     INTEGER NOT NULL REFERENCES topics(id),     status
TEXT NOT NULL DEFAULT 'todo'                  CHECK (status IN ('todo',
'in_progress', 'complete')),     archived_at  TEXT,     created_at   TEXT NOT NULL
DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ','now')),     updated_at   TEXT NOT NULL
DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ','now')) );  CREATE INDEX idx_tasks_topic_id
ON tasks(topic_id); CREATE INDEX idx_tasks_status      ON tasks(status); CREATE INDEX
idx_tasks_due_date    ON tasks(due_date); CREATE INDEX idx_tasks_archived_at ON
tasks(archived_at);
```